ruina_do_jogador

October 9, 2026

0.1 Simulação da ruína do jogador

Primeiro, definimos uma função `ruina(i, N, p)` que realiza uma partida, onde `i`, `N`, e `p` são o saldo inicial, o objetivo e a probabilidade de vitória, respectivamente. Começando o jogo com i moedas no tempo $T = 0$, continuamos jogando enquanto $x \neq 0$ e $x \neq N$. Usamos o módulo `np.random.binomial(n, p)` para jogar a moeda uma vez (`n = 1` com probabilidade de vitória `p`). Essa função retorna o número de sucessos, neste caso, `0` ou `1`. Se retorna `1`, então adiciona 1 ao saldo. Caso contrário, subtrai 1. A função `ruina()` retorna dois valores: um booleano (`True` se o jogador atingiu `N` e `False` caso contrário) e o número de lançamentos da moeda.

[2]:
```
import numpy as np

def ruina(i, N, p):
    x = i
    T = 0

    while x != 0 and x != N:
        passo = np.random.binomial(1, p)

        if passo == 1:
            x += 1
        else:
            x -= 1

        T += 1

    sucesso = (x == N)

    return sucesso, T
```

0.1.1 Função que simula um jogo

Escrevemos a função `simulacao_ruina(i, N, p, n_sim)` que realiza a jogatina `n_sim` vezes e retorna a probabilidade de sucesso (a razão entre o número de sucessos e o número de simulações), o tempo médio de uma jogatina (a média do número de lançamentos de moeda entre todas as simulações) e o erro entre a probabilidade de sucesso exata e a observada nas simulações.

[3]:
```
def simulacao_ruina(i, N, p, n_sim=1000):
    sucessos = 0
```

```
    tempos = []

    for _ in range(n_sim):
        sucesso, T = ruina(i, N, p)

        if sucesso:
            sucessos += 1

        tempos.append(T)

    prob_sucesso = sucessos / n_sim
    tempo_medio = np.mean(tempos)

    # calcula o erro entre a probabilidade da simulacao e a probabilidade exata␣
↪(solucao da ruina)
    if p == 0.5:
        prob_exata = i/N
        erro = np.abs(prob_sucesso - prob_exata)
    else:
        q = 1 - p
        prob_exata = (1 - (q/p)**i) / (1 - (q/p)**N)
        erro = np.abs(prob_sucesso - prob_exata)

    return prob_sucesso, tempo_medio, erro
```

Realizamos uma jogatina, com saldo inicial $i = 4$, objetivo $N = 10$ e probabilidade de vitória $p = 0.5$.

[4]:
```
prob, tempo, erro = simulacao_ruina(i=4, N=10, p=0.5, n_sim=2000)

print("Probabilidade de sucesso:", prob)
print("Tempo médio:", tempo)
print(f"Erro: {erro:.2e}")
```

```
Probabilidade de sucesso: 0.395
Tempo médio: 23.308
Erro: 5.00e-03
```

0.1.2 Simulação N=10, p=0.5, i=1:9

Agora, mantemos os parâmetros $N = 10$ e $p = 0.5$ e varremos i (saldo inicial) de 1 a 9.

[5]:
```
import tabulate
import matplotlib.pyplot as plt

N = 10
p = 0.5
q = 1 - p
```

```
dados = []

for i in range(1, 10):
    prob, tempo, erro = simulacao_ruina(i, N, p, n_sim=2000)
    dados.append([i, prob, tempo, erro])

print(tabulate.tabulate(dados, headers=['Saldo inicial', 'Probabilidade de␣
 ↪sucesso', 'Tempo médio', 'Erro'], tablefmt='github', floatfmt=['','.4f', '.
 ↪2f', '.2e']))

# extrai dados da tabela para plotar graficos
i = [linha[0] for linha in dados]
prob = [linha[1] for linha in dados]
tempo = [linha[2] for linha in dados]
erro = [linha[3] for linha in dados]

prob_exata = [i_val / 10 for i_val in i]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
fig.suptitle(r'jogo com $N=10$ e $p=0.5$')

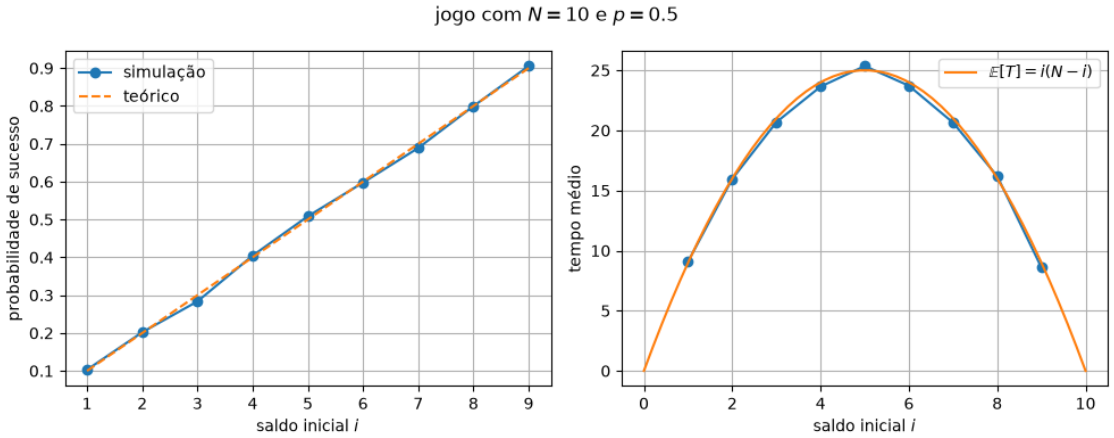
ax1.plot(i, prob, 'o-', label='simulação')
ax1.plot(i, prob_exata, '--', label='teórico')
ax1.set_xlabel('saldo inicial $i$')
ax1.set_ylabel('probabilidade de sucesso')
ax1.legend()
ax1.grid()

ii = np.linspace(0, 10)
ax2.plot(i, tempo, 'o-')
ax2.plot(ii, ii*(N-ii), label=r'$\mathbb{E}[T]=i(N-i)$')
ax2.set_xlabel('saldo inicial $i$')
ax2.set_ylabel('tempo médio')
ax2.grid()
ax2.legend()

plt.tight_layout()
plt.show()
```

Saldo inicial	Probabilidade de sucesso	Tempo médio	Erro
1	0.1030	9.09	3.00e-03
2	0.2020	15.96	2.00e-03
3	0.2835	20.68	1.65e-02
4	0.4045	23.64	4.50e-03
5	0.5085	25.35	8.50e-03
6	0.5975	23.72	2.50e-03
7	0.6890	20.64	1.10e-02

```
|             8 |                  0.7995 |         16.19 | 5.00e-04 |
|             9 |                  0.9055 |          8.60 | 5.50e-03 |
```



0.1.3 Conjectura da esperança para jogos simétricos

Do segundo gráfico, vemos que a esperança (ou o tempo médio) do tempo de parada se comporta como uma função quadrática do saldo inicial: uma parabóla com concavidade voltada para baixo que possui zeros em $i = 0$ e $i = N$. Como $\mathbb{E}_0[T] = 0$ e $\mathbb{E}_N[T] = 0$, conjecturamos que, para $p = q = 1/2$, deve valer

$$\mathbb{E}[T] = i(N - i).$$

De fato, esse modelo fornece os valores

$$9, 16, 21, 24, 25, 24, 21, 16, 9,$$

que estão de acordo com os resultados obtidos na simulação.

0.1.4 Simulação N=10, p=0.3, i=1:9

Alterando a probabilidade de vitória para $p = 0.3$, obtemos

[6]:
```
N = 10
p = 0.3
q = 1 - p
dados = []

for i in range(1, 10):
    prob, tempo, erro = simulacao_ruina(i, N, p, n_sim=2000)
    dados.append([i, prob, tempo, erro])

print(tabulate.tabulate(dados, headers=['Saldo inicial', 'Probabilidade de␣
 ↪sucesso', 'Tempo médio', 'Erro'], tablefmt='github', floatfmt=['','.4f', '.
 ↪2f', '.2e']))
```

```python
# extrai dados da tabela para plotar graficos
i = [linha[0] for linha in dados]
prob = [linha[1] for linha in dados]
tempo = [linha[2] for linha in dados]
erro = [linha[3] for linha in dados]

prob_exata = [(1 - (q/p)**i_val) / (1 - (q/p)**N) for i_val in i]

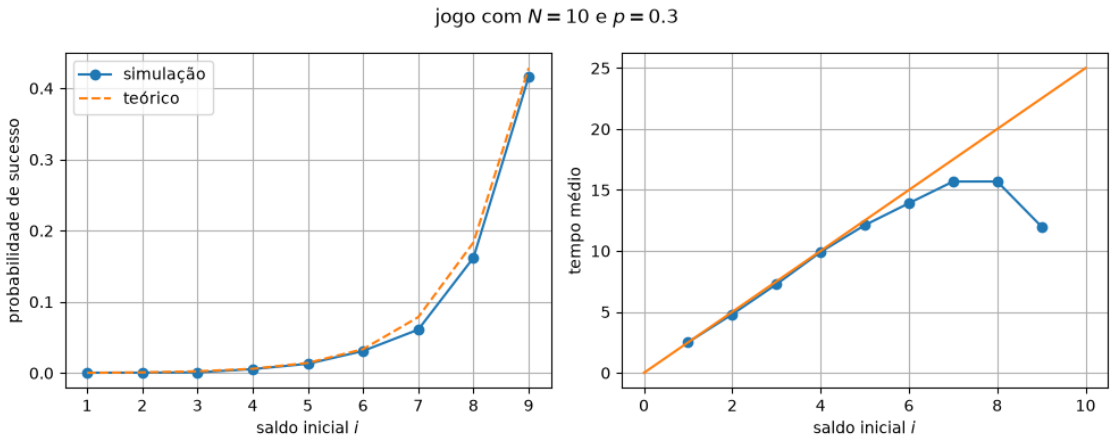
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
fig.suptitle(r'jogo com $N=10$ e $p=0.3$')

ax1.plot(i, prob, 'o-', label='simulação')
ax1.plot(i, prob_exata, '--', label='teórico')
ax1.set_xlabel('saldo inicial $i$')
ax1.set_ylabel('probabilidade de sucesso')
ax1.legend()
ax1.grid()

ii = np.linspace(0, 10)
ax2.plot(i, tempo, 'o-')
ax2.plot(ii, 2.5*ii, label=r'$\mathbb{E}[T]=2.5i$')
ax2.set_xlabel('saldo inicial $i$')
ax2.set_ylabel('tempo médio')
ax2.grid()

plt.tight_layout()
plt.show()
```

Saldo inicial	Probabilidade de sucesso	Tempo médio	Erro
1	0.0005	2.51	2.21e-04
2	0.0005	4.80	4.29e-04
3	0.0010	7.26	1.45e-03
4	0.0055	9.91	4.89e-04
5	0.0130	12.11	1.25e-03
6	0.0310	13.91	2.53e-03
7	0.0610	15.69	1.75e-02
8	0.1620	15.69	2.15e-02
9	0.4160	12.01	1.25e-02



0.1.5 Conjectura da esperança para jogos assimétricos

Agora, como $p \neq q$, o jogo perde a simetria e o gráfico da esperança em função do saldo inicial se assemelha a uma reta para quase todo saldo, exceto para os valores mais próximos do objetivo. Neste caso, como $p < q$, é natural esperar que a jogatina leve mais tempo para acabar conforme o jogador começa com um saldo mais alto. Isso porque, embora ele esteja mais próximo do objetivo, a probabilidade de vitória em cada lançamento é menor que a de derrota e, uma vez que o jogador começa a perder, a probabilidade de atingir o objetivo é menor e a jogatina se arrasta no tempo. No entanto, ainda temos $\mathbb{E}_N[T] = 0$, então o comportamento da função muda bastante perto dessa fronteira: se o jogador começa a jogatina com saldo inicial muito perto do objetivo, a chance de ele atingir o objetivo é alta e, portanto, o tempo de espera diminui.

Para estimar o comportamento linear da esperança, computamos a declividade m da reta que passa pelos pontos (1, 2.59) e (2, 5.12) e obtemos $m = 2.53$. Plotamos o gráfico da reta $\mathbb{E}[T] = 2.5i$, que parece ajustar bem o conjunto de pontos até $i = 7$. Notamos que

$$\frac{1}{q-p} = 2.5,$$

então temos

$$\mathbb{E}[T] = \frac{i}{q-p}$$

para pelo menos um subconjunto dos saldos iniciais.

[7]:
```
m = (5.12 - 2.59)
print(m)

print(1/(q-p))
```

```
2.5300000000000002
2.5
```

0.1.6 Simulação N=100, p=0.3, i=1:99

Com $N = 100$, $p = 0.3$, varremos i de 1 a 99:

[ ]:
```
N = 100
p = 0.7
q = 1 - p

dados = []

for i in range(1, 100):
    prob, tempo, erro = simulacao_ruina(i, N, p)
    dados.append([i, prob, tempo, erro])

print(tabulate.tabulate(dados, headers=['Saldo inicial', 'Probabilidade de␣
 ↪sucesso', 'Tempo médio', 'Erro'], tablefmt='github', floatfmt=['','.4f', '.
 ↪2f', '.2e']))

# extrai dados da tabela para plotar graficos
i = [linha[0] for linha in dados]
prob = [linha[1] for linha in dados]
tempo = [linha[2] for linha in dados]
erro = [linha[3] for linha in dados]

prob_exata = [(1 - (q/p)**i_val) / (1 - (q/p)**N) for i_val in i]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
fig.suptitle(r'jogo com $N=100$ e $p=0.3$')

ax1.plot(i, prob, 'o-', label='simulação')
ax1.plot(i, prob_exata, '--', label='teórico')
ax1.set_xlabel('saldo inicial $i$')
ax1.set_ylabel('probabilidade de sucesso')
ax1.legend()
ax1.grid()

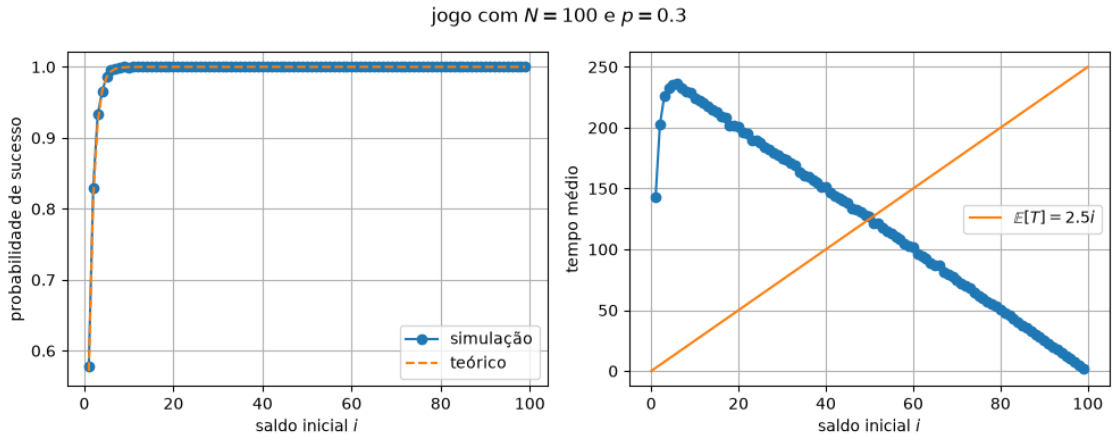
ii = np.linspace(0, 100)
ax2.plot(i, tempo, 'o-')
ax2.plot(ii, ii/(q-p), label=r'$\mathbb{E}[T]=2.5i$')
ax2.set_xlabel('saldo inicial $i$')
ax2.set_ylabel('tempo médio')
ax2.legend()
ax2.grid()

plt.tight_layout()
plt.show()
```

Saldo inicial	Probabilidade de sucesso	Tempo médio	Erro
1	0.5780	142.92	6.57e-03
2	0.8300	202.79	1.37e-02
3	0.9330	225.91	1.17e-02
4	0.9650	232.24	1.26e-03
5	0.9870	235.32	1.46e-03
6	0.9960	235.92	2.20e-03
7	0.9980	232.87	6.56e-04
8	0.9990	230.16	1.38e-04
9	1.0000	228.91	4.88e-04
10	0.9990	224.28	7.91e-04
11	1.0000	222.01	8.96e-05
12	1.0000	220.83	3.84e-05
13	1.0000	217.82	1.65e-05
14	1.0000	214.84	7.05e-06
15	1.0000	212.64	3.02e-06
16	1.0000	209.50	1.30e-06
17	1.0000	208.73	5.55e-07
18	1.0000	201.82	2.38e-07
19	1.0000	201.80	1.02e-07
20	1.0000	200.65	4.37e-08
21	1.0000	196.19	1.87e-08
22	1.0000	195.60	8.03e-09
23	1.0000	190.01	3.44e-09
24	1.0000	189.43	1.47e-09
25	1.0000	187.84	6.32e-10
26	1.0000	184.06	2.71e-10
27	1.0000	182.29	1.16e-10
28	1.0000	179.05	4.97e-11
29	1.0000	177.89	2.13e-11
30	1.0000	174.88	9.13e-12
31	1.0000	173.79	3.91e-12
32	1.0000	170.79	1.68e-12
33	1.0000	169.03	7.19e-13
34	1.0000	163.53	3.08e-13
35	1.0000	160.92	1.32e-13
36	1.0000	159.62	5.66e-14
37	1.0000	157.39	2.42e-14
38	1.0000	154.84	1.04e-14
39	1.0000	151.52	4.44e-15
40	1.0000	151.63	1.89e-15
41	1.0000	147.09	7.77e-16
42	1.0000	143.54	3.33e-16
43	1.0000	142.14	1.11e-16
44	1.0000	140.50	1.11e-16
45	1.0000	138.25	0.00e+00
46	1.0000	133.98	0.00e+00

47	1.0000	132.72	0.00e+00
48	1.0000	130.75	0.00e+00
49	1.0000	128.01	0.00e+00
50	1.0000	126.64	0.00e+00
51	1.0000	121.69	0.00e+00
52	1.0000	121.07	0.00e+00
53	1.0000	117.64	0.00e+00
54	1.0000	115.33	0.00e+00
55	1.0000	112.67	0.00e+00
56	1.0000	110.65	0.00e+00
57	1.0000	108.05	0.00e+00
58	1.0000	104.53	0.00e+00
59	1.0000	102.54	0.00e+00
60	1.0000	101.54	0.00e+00
61	1.0000	96.63	0.00e+00
62	1.0000	94.79	0.00e+00
63	1.0000	92.11	0.00e+00
64	1.0000	88.93	0.00e+00
65	1.0000	87.29	0.00e+00
66	1.0000	86.63	0.00e+00
67	1.0000	81.12	0.00e+00
68	1.0000	79.27	0.00e+00
69	1.0000	77.23	0.00e+00
70	1.0000	74.72	0.00e+00
71	1.0000	72.31	0.00e+00
72	1.0000	70.35	0.00e+00
73	1.0000	68.02	0.00e+00
74	1.0000	64.68	0.00e+00
75	1.0000	62.06	0.00e+00
76	1.0000	59.58	0.00e+00
77	1.0000	56.64	0.00e+00
78	1.0000	55.10	0.00e+00
79	1.0000	53.04	0.00e+00
80	1.0000	50.08	0.00e+00
81	1.0000	47.96	0.00e+00
82	1.0000	45.68	0.00e+00
83	1.0000	42.85	0.00e+00
84	1.0000	40.28	0.00e+00
85	1.0000	37.56	0.00e+00
86	1.0000	35.21	0.00e+00
87	1.0000	33.06	0.00e+00
88	1.0000	29.76	0.00e+00
89	1.0000	27.67	0.00e+00
90	1.0000	24.99	0.00e+00
91	1.0000	22.55	0.00e+00
92	1.0000	19.80	0.00e+00
93	1.0000	17.74	0.00e+00
94	1.0000	14.89	0.00e+00

```
|              95 |                  1.0000 |         12.51 | 0.00e+00 |
|              96 |                  1.0000 |         10.00 | 0.00e+00 |
|              97 |                  1.0000 |          7.76 | 0.00e+00 |
|              98 |                  1.0000 |          4.86 | 0.00e+00 |
|              99 |                  1.0000 |          2.43 | 0.00e+00 |
```



0.1.7 Simulação N=100, p=0.1, i=1:99

Colocando $p = 0.1$, a probabilidade de vitória fica muito baixa.

[11]:
```
N = 100
p = 0.1
q = 1 - p

dados = []

for i in range(1, 100):
    prob, tempo, erro = simulacao_ruina(i, N=100, p=0.1)
    dados.append([i, prob, tempo, erro])

print(tabulate.tabulate(dados, headers=['Saldo inicial', 'Probabilidade de␣
 ↪sucesso', 'Tempo médio', 'Erro'], tablefmt='github', floatfmt=['','.4f', '.
 ↪2f', '.2e']))

# extrai dados da tabela para plotar graficos
i = [linha[0] for linha in dados]
prob = [linha[1] for linha in dados]
tempo = [linha[2] for linha in dados]
erro = [linha[3] for linha in dados]

prob_exata = [(1 - (q/p)**i_val) / (1 - (q/p)**N) for i_val in i]
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
fig.suptitle(r'jogo com $N=100$ e $p=0.1$')

ax1.plot(i, prob, 'o-', label='simulação')
ax1.plot(i, prob_exata, '--', label='teórico')
ax1.set_xlabel('saldo inicial $i$')
ax1.set_ylabel('probabilidade de sucesso')
ax1.legend()
ax1.grid()

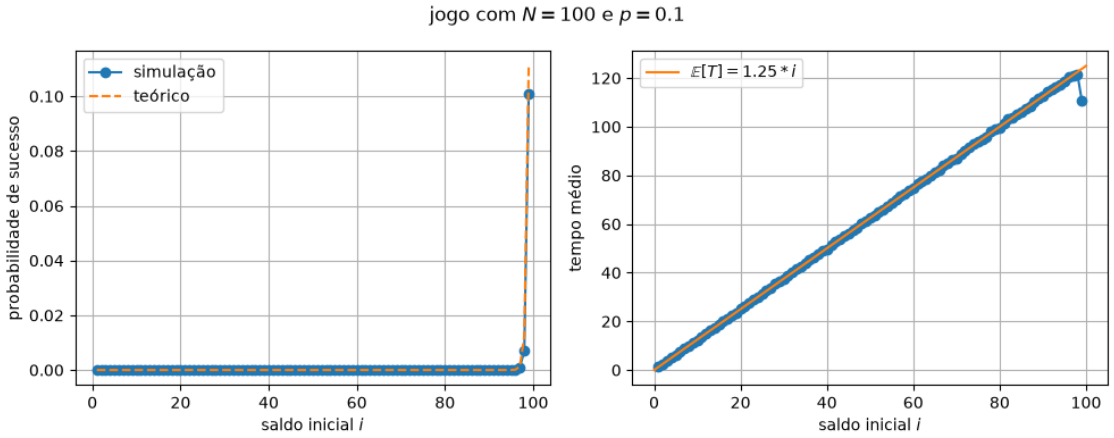
ii = np.linspace(0, 100)
ax2.plot(i, tempo, 'o-')
ax2.plot(ii, ii/(q-p), label=r'$\mathbb{E}[T]=1.25*i$')
ax2.set_xlabel('saldo inicial $i$')
ax2.set_ylabel('tempo médio')
ax2.legend()
ax2.grid()

plt.tight_layout()
plt.show()
```

Saldo inicial	Probabilidade de sucesso	Tempo médio	Erro
1	0.0000	1.28	3.01e-95
2	0.0000	2.42	3.01e-94
3	0.0000	3.71	2.74e-93
4	0.0000	4.98	2.47e-92
5	0.0000	6.31	2.22e-91
6	0.0000	7.59	2.00e-90
7	0.0000	8.77	1.80e-89
8	0.0000	9.91	1.62e-88
9	0.0000	11.23	1.46e-87
10	0.0000	12.38	1.31e-86
11	0.0000	13.75	1.18e-85
12	0.0000	15.08	1.06e-84
13	0.0000	16.26	9.57e-84
14	0.0000	17.42	8.61e-83
15	0.0000	18.78	7.75e-82
16	0.0000	19.90	6.98e-81
17	0.0000	21.25	6.28e-80
18	0.0000	22.47	5.65e-79
19	0.0000	23.56	5.09e-78
20	0.0000	25.10	4.58e-77
21	0.0000	26.31	4.12e-76
22	0.0000	27.54	3.71e-75
23	0.0000	28.78	3.34e-74
24	0.0000	30.01	3.00e-73
25	0.0000	31.34	2.70e-72

26	0.0000	32.58	2.43e-71
27	0.0000	33.76	2.19e-70
28	0.0000	35.45	1.97e-69
29	0.0000	36.26	1.77e-68
30	0.0000	37.33	1.60e-67
31	0.0000	39.00	1.44e-66
32	0.0000	40.10	1.29e-65
33	0.0000	41.42	1.16e-64
34	0.0000	42.36	1.05e-63
35	0.0000	43.87	9.42e-63
36	0.0000	45.41	8.48e-62
37	0.0000	46.28	7.63e-61
38	0.0000	47.61	6.87e-60
39	0.0000	49.00	6.18e-59
40	0.0000	49.73	5.56e-58
41	0.0000	51.32	5.01e-57
42	0.0000	52.71	4.51e-56
43	0.0000	53.90	4.06e-55
44	0.0000	55.18	3.65e-54
45	0.0000	56.11	3.29e-53
46	0.0000	57.31	2.96e-52
47	0.0000	58.44	2.66e-51
48	0.0000	60.14	2.40e-50
49	0.0000	61.36	2.16e-49
50	0.0000	62.47	1.94e-48
51	0.0000	63.69	1.75e-47
52	0.0000	65.18	1.57e-46
53	0.0000	66.00	1.41e-45
54	0.0000	67.37	1.27e-44
55	0.0000	68.75	1.15e-43
56	0.0000	70.08	1.03e-42
57	0.0000	71.49	9.28e-42
58	0.0000	72.48	8.35e-41
59	0.0000	73.63	7.52e-40
60	0.0000	74.91	6.77e-39
61	0.0000	76.59	6.09e-38
62	0.0000	77.64	5.48e-37
63	0.0000	78.40	4.93e-36
64	0.0000	79.76	4.44e-35
65	0.0000	81.34	3.99e-34
66	0.0000	82.29	3.60e-33
67	0.0000	84.03	3.24e-32
68	0.0000	85.10	2.91e-31
69	0.0000	86.54	2.62e-30
70	0.0000	86.84	2.36e-29
71	0.0000	88.91	2.12e-28
72	0.0000	90.25	1.91e-27
73	0.0000	91.54	1.72e-26

74	0.0000	93.01	1.55e-25
75	0.0000	93.82	1.39e-24
76	0.0000	94.92	1.25e-23
77	0.0000	95.65	1.13e-22
78	0.0000	97.96	1.02e-21
79	0.0000	99.01	9.14e-21
80	0.0000	99.69	8.23e-20
81	0.0000	101.14	7.40e-19
82	0.0000	103.07	6.66e-18
83	0.0000	103.93	6.00e-17
84	0.0000	105.04	5.40e-16
85	0.0000	106.02	4.86e-15
86	0.0000	107.39	4.37e-14
87	0.0000	108.40	3.93e-13
88	0.0000	110.30	3.54e-12
89	0.0000	111.62	3.19e-11
90	0.0000	112.79	2.87e-10
91	0.0000	114.56	2.58e-09
92	0.0000	115.18	2.32e-08
93	0.0000	116.14	2.09e-07
94	0.0000	117.02	1.88e-06
95	0.0000	118.54	1.69e-05
96	0.0000	120.42	1.52e-04
97	0.0010	121.18	3.72e-04
98	0.0070	121.64	5.35e-03
99	0.1010	110.93	1.01e-02



[10]:
```
m = 1/(q-p)
print(m)
```

```
1.25
```

0.1.8 Relação de recorrência para a esperança

Podemos estabelecer uma relação de recorrência para $f_i = \mathbb{E}_t[T]$ no caso geral. Seja

$$f_i = \mathbb{E}_i[T], \quad 1 \leq i \leq N-1,$$

o tempo esperado até o fim do jogo dado que o saldo inicial é i, com condições de fronteira $f_0 = f_N = 0$. Suponha que um jogador comece com saldo i, $1 \leq i \leq N-1$ e considere os eventos

- $A =$ ganha a primeira rodada
- $B =$ perde a primeira rodada

com probabilidades $P(A) = p$ e $P(B) = q$. É claro que A e B formam uma partição do espaço amostral para a primeira rodada, logo

$$\mathbb{E}_i[T] = p\mathbb{E}_i[T|A] + q\mathbb{E}_i[T|B].$$

Perceba que se o jogador ganha a primeira rodada, o saldo fica $i+1$ e o tempo aumenta em uma unidade. Então é como se o jogo recomeçasse com saldo $X_0 = i+1$ e tempo de espera acrescido de 1, ou seja,

$$\mathbb{E}_i[T|A] = \mathbb{E}_{i+1}[T] + 1 = f_{i+1} + 1.$$

Analogamente, caso ele perca, temos

$$\mathbb{E}_i[T|B] = \mathbb{E}_{i-1}[T] + 1 = f_{i-1} + 1,$$

o que nos leva à seguinte relação de recorrência:

$$\begin{aligned} f_i &= p(f_{i+1} + 1) + q(f_{i-1} + 1) \\ &= (p+q) + pf_{i+1} + qf_{i-1} \\ &= 1 + pf_{i+1} + qf_{i-1} \end{aligned}$$